

Definire la Complessità di Spazio per le Macchine di Turing

def La complessità di Spazio di una TM Decisore M è una funzione $S: \mathbb{N} \rightarrow \mathbb{N}$
t.c. $S(n) = \max_{|x|=n} \{ \# \text{celle di nastro scritte da } M(x) \}$

Enunciare e Dimostrare il Teorema di Savitch

Teorema: $NL \subseteq P$ e $NL \subseteq SPACE(\log^2 n)$

Dim: Sia $L \in NL$. Dobbiamo mostrare che $L \in P$ ed $L \in SPACE(\log^2 n)$

$NL \subseteq P$ Siccome $L \in NL$ si ha che $\exists N \text{ TM } N$ t.c. $L(N) = L$ ed N richiede log-space.

Possiamo quindi costruire un grafo $G_{N,x}$ che rappresenta l'albero di computazione di $N(x)$.
Avremo quindi che ogni nodo di $G_{N,x}$ è una configurazione di N , ed avremo un arco $C \rightarrow C'$ sse. è possibile andare dalla configurazione C a C' direttamente.

Abbiamo quindi ridotto il problema a trovare un percorso che va dalla conf. iniziale ad una qualsiasi conf. accettabile. Per renderlo una vera istanza di PATH dobbiamo fare in modo che N abbia un solo stato accettabile (C_{acc}). Ad esempio svuotando il nastro e spostando tutte le testine a sx.

Inoltre, siccome N è log-space, si ha che il #configurazioni possibili per $N(x)$ è $2^{O(\log n)} = O(n^k)$. Dunque anche il grafo ha $\# \text{nodi} = O(n^k) = \text{POLINOMIALE}$.

Possiamo quindi creare una TM Deterministica M t.c.

- Su input $\langle N, x \rangle$ costruisce il grafo $G_{N,x}$ e lo scrive sul nastro di LAVORO
- Controlla se $\langle C_{START}, C_{acc}, G_{N,x} \rangle \in \text{PATH}$

E' evidente che $L(H) = L(N)$ poiché $M(\langle N, x \rangle) = ACC \Leftrightarrow \exists C_{START} \rightsquigarrow C_{ACC} \text{ in } G_{N,x} \Leftrightarrow N(x) = ACC$

Inoltre, poiché $G_{N,x}$ ha un numero di nodi POLINOMIALE e $PATH \in P$, si ha che:

- Costruire $G_{N,x}$ richiede tempo polinomiale
- Scrivere $G_{N,x}$ sul nastro richiede tempo polinomiale
- Controllare $\langle C_{START}, C_{ACC}, G_{N,x} \rangle \in PATH$ richiede tempo polinomiale

Concludiamo quindi che $L(H) = L(N) = L \in P$.

$NL \subseteq SPACE(\log^2 n)$ Dobbiamo mostrare che $\exists M \in TM$ Deterministica t.c. $L(M) = L$ ed M richiede spazio $O(\log^2 n)$

Nella pratica possiamo costruire la stessa identica TM M di prima, ma stavolta NON possiamo scrivere tutto il grafo $G_{N,x}$ sul nastro. (Come abbiamo visto prima, $G_{N,x}$ ha un numero di nodi polinomiale. Quindi scrivere tutto $G_{N,x}$ richiederebbe spazio polinomiale. A noi serve $O(\log^2 n)$)

Utilizziamo quindi l'algoritmo di Savitch, ovvero la funzione ricorsiva $PATH(C_x, C_y, k)$ che, in spazio $O(\log^2 n)$, ci dice se esiste un percorso $C_x \rightsquigarrow C_y$ lungo 2^k .

Funzionamento:

- se $k=0$:
 - ritorna 'SI' se $C_x = C_y$ oppure $(C_x, C_y) \in E$
 - ritorna 'NO' altrimenti
- se $k > 0$ effettua le chiamate ricorsive. $\forall C_w \in V$ chiama:

$$PATH(C_x, C_w, k-1) \quad \wedge \quad PATH(C_w, C_y, k-1)$$

ritorna "SI" solo se entrambe ritornano

Lanciamo quindi $\text{PATH}(C_{\text{START}}, C_{\text{ACC}}, O(\log n))$. Poiché ogni chiamata ricorsiva costa $O(\log n)$ e facciamo $O(\log n)$ chiamate, avremo che H richiede spazio $O(\log^2 n)$

Dunque $L(H) = L$ ed H richiede spazio $O(\log^2 n) \Rightarrow L \in \text{SPACE}(\log^2 n)$

Inoltre non avendo a disposizione il grafo $G_{n,x}$ non possiamo andare a cercare direttamente gli archi che formano il percorso $C_x \rightsquigarrow C_w \dots$ dobbiamo trovare un altro modo. In particolare ci basta recuperare la configurazione $C = w0 \boxed{q} 2K; i$ dal nastro di input e controllare cosa dice la $\delta_n(q, x_i)$.

Tutto il teorema e la dimostrazione possono essere generalizzati

Teorema: $\text{NSPACE}(f(n)) \subseteq \text{DTIME}(2^{O(f(n))})$ e $\text{NSPACE}(f(n)) \subseteq \text{DSPACE}(f^2(n))$

Si definiscano le classi di Complessità PSPACE ed NPSPACE.

Si dimostri che $PSPACE = NPSPACE$

$$PSPACE = \bigcup_{k \in \mathbb{N}} SPACE(n^k) = \{L : \exists H \in TM \text{ t.c. } L(H) = L \text{ ed } H \text{ richiede spazio } O(n^k)\}$$

$$NPSPACE = \bigcup_{k \in \mathbb{N}} NSPACE(n^k) = \{L : \exists N \in NTH \text{ t.c. } L(N) = L \text{ ed } N \text{ richiede spazio } O(n^k)\}$$

Mostriamo ora che $PSPACE = NPSPACE$ usando la doppia inclusione:

$PSPACE \subseteq NPSPACE$

Mostriamo che $\forall L \in PSPACE$ si ha anche $L \in NPSPACE$.

E' quasi banale. $L \in PSPACE$ significa che $\exists H \in TM$ t.c. $L(H) = L$ ed H richiede spazio polinomiale.

Poiché ogni TM è anche una NTH con un singolo ramo di computazione basta prendere una NTH $N = H$ ed avremo che $\exists N \in NTH$ t.c. $L(N) = L$ ed N richiede spazio polinomiale
 $\Rightarrow L \in NPSPACE$.

$NPSPACE \subseteq PSPACE$

Mostriamo che $\forall L \in NPSPACE$ si ha anche $L \in PSPACE$.

Per il teorema di Savitch si ha che $NSPACE(f(n)) \subseteq SPACE(f^2(n))$. Applicando questo teorema al nostro caso otteniamo che:

$$NPSPACE = NSPACE(n^k) \subseteq SPACE(n^{2k}) = PSPACE$$

Si definiscano i concetti di Riducibilità Polinomiale, NP Completezza e CoNP Completezza

Siano A, B linguaggi. Diremo che $A \leq_m^p B$ se esiste una funzione calcolabile $R: \Sigma^* \rightarrow \Sigma^*$ di tempo polinomiale tale che $\forall x \in \Sigma^* \quad x \in A \Leftrightarrow R(x) \in B$

Inoltre si ha che un linguaggio L è NP-Completo se $L \in \text{NP}$ e $\forall A \in L \quad A \leq_m^p L$

Analogamente, definiamo $\text{coNP} = \{L : \bar{L} \in \text{NP}\}$, ed avremo che un linguaggio L è coNP-Completo se $L \in \text{coNP}$ e $\forall A \in \text{coNP} \quad A \leq_m^p L$

Dimostrare che $3COL \leq_m^P SAT$

Ovvero, dato un grafo $G = (V, E)$ 3-colorabile dobbiamo costruire una formula ϕ_G t.e. ϕ_G soddisfacibile ssc. G è 3-col.

La formula avrà $2n$ variabili: $x_1, x_1', x_2, x_2', \dots, x_n, x_n'$ dove $n = |V|$.

Usiamo due variabili per ogni nodo, x_i ed x_i' , poiché questi agiscono da "bit" che ci permettono di codificare 3 colori:

x_i	x_i'	C_i
0	0	Red
0	1	Blue
1	0	Yellow
1	1	INVALID

Nella formula, per ogni arco $(i, j) \in E$, codifichiamo la regola ^{logica in cui possiamo usare solo $\wedge, \vee, \neg$} SAT di avere colori diversi:

$$(x_i, x_i') \neq (x_j, x_j') \iff \neg(x_i \leftrightarrow x_j \wedge x_i' \leftrightarrow x_j')$$

A questo punto traduciamo le " $\leftrightarrow$ " in " $\wedge$ ", " $\vee$ ", " $\neg$ " e otteniamo una nuova formula equivalente $\phi_{i,j}$ che controlla la coppia di nodi $(i, j) \in E$... Ma noi dobbiamo controllare tutto il grafo! Dunque basta mettere in " $\wedge$ " tutte le $\phi_{i,j}$ per ogni coppia di nodi (i, j) .

$$\phi_G(.) = \bigwedge_{(i,j) \in E} \phi_{i,j}(.)$$

E aggiungere la formula che VIETA di avere colore "11".

$$\phi_i = (\neg x_i \vee \neg x_i')$$

Combiniamo le due cose ed otteniamo:

$$\phi_G(.) = \bigwedge_{(i,j) \in E} \phi_{i,j}(.) \wedge \bigwedge_{i \in V} \phi_i$$

Dim di Correttezza: Mostriamo che $\langle G \rangle \in 3\text{-COL} \iff R(\langle G \rangle) = \phi_G \in \text{SAT}$

$\Rightarrow$ Se $\langle G \rangle \in 3\text{-COL}$ si ha che $\exists$ 3-colorazione per G . Questo significa che tutti i vicini hanno colori diversi. Dunque $\forall (i,j) \in E$ si ha sicuramente $(x_i, x'_i) \neq (x_j, x'_j)$.
Dunque $\forall (i,j) \in E$ la formula ϕ_{ij} è soddisfatta.

Inoltre, se $\langle G \rangle \in 3\text{-COL}$, si ha sicuramente che utilizza solo 3 colori. Dunque anche ϕ_i è soddisfatta $\forall i$.

Poiché ϕ_G è solo un AND delle due formule sopra otteniamo che: $\langle \phi_G \rangle = R(\langle G \rangle) \in \text{SAT}$.

$\Leftarrow R(\langle G \rangle) = \langle \phi_G \rangle \notin \text{SAT} \Rightarrow \langle G \rangle \notin 3\text{-COL}$

Se $\langle \phi_G \rangle \notin \text{SAT}$ significa che $\phi_G = 0$. Per come abbiamo costruito R , si ha che $\phi_G = 0$ solo se G ha almeno 2 nodi vicini con lo stesso colore oppure G utilizza 4 colori. In entrambi i casi è evidente che $\langle G \rangle \notin 3\text{-COL}$.

Si dimostri che un linguaggio L è NP Completo sse. $\bar{L}$ è CoNP Completo

$\Rightarrow$ Dobbiamo mostrare che L NP-Completo $\Rightarrow \bar{L}$ coNP Completo

Innanzitutto si nota che $\bar{L} \in \text{coNP}$,^① poiché $\overline{(\bar{L})} = L \in \text{NP}$ per ipotesi. Dobbiamo quindi far vedere che $\forall A \in \text{coNP} \quad A \leq_m^P \bar{L}$.

Sfruttando l'ipotesi iniziale che L è NP-Completo possiamo con certezza dire che $\forall A \in \text{NP} \quad A \leq_m^P L$. A questo punto possiamo negare da entrambi i lati ed ottenere che

$$\forall A \in \text{NP} \quad \bar{A} \leq_m^P \bar{L}.$$

Ma siccome $A \in \text{NP}$, si ha che $\bar{A} \in \text{coNP}$. Dunque, prendendo $A' = \bar{A}$, si ha che

$$\textcircled{2} \quad \forall A' \in \text{coNP} \quad A' \leq_m^P \bar{L}$$

Dunque per $\textcircled{1}$ e $\textcircled{2}$ si ha che $\bar{L}$ è coNP Completo

$\Leftarrow$ Dobbiamo mostrare che $\bar{L}$ coNP Completo $\Rightarrow L$ NP-Completo

Speculare a quello sopra.

Definire la Complessità di Tempo per le Macchine di Turing

La complessità di tempo di una TM M DECISORE è una funzione $T: \mathbb{N} \rightarrow \mathbb{N}$ tale.

$$T(n) = \max \{ \# \text{passi richiesti da } M(x) \} \text{ con } x \in \Sigma^* \text{ ed } n = |x|$$

Enunciare e Dimostrare il Teorema di Gerarchia di Tempo

Teorema: Sia $t_1(n) \gg n$ e $t_2(n) = \Omega(t_1(n) \log t_1(n))$ entrambi tempo costruibili.
Allora $\exists L \in \text{DTIME}(t_2(n))$ ma $L \notin \text{DTIME}(t_1(n))$

Dim. Siano $t_1(n) \ll t_{1.5}(n) \ll t_2(n)$. Vogliamo mostrare che $\exists L \in \text{DTIME}(t_2(n))$ MA $L \notin \text{DTIME}(t_1(n))$. A tale scopo definiamo una TM D tale. D richiede tempo $t_2(n)$ MA $L(D)$ non può essere deciso in tempo $t_1(n)$. Nella pratica, la TM D :

- Su input $\langle M \rangle$ con $M \in \text{TM}$
- Simula $M(\langle M \rangle)$
 - Se $M(\langle M \rangle)$ termina prima di $t_{1.5}(n)$ passi allora D nega il risultato
Cioè $D(\langle M \rangle) = \text{ACC} \Leftrightarrow M(\langle M \rangle) = \text{REJ}$ e viceversa
 - Se $M(\langle M \rangle)$ impiega più di $t_{1.5}(n)$ passi, allora $D(\langle M \rangle) = \text{ACC}$.

Costo Temporale: Poiché $D(\langle M \rangle)$ si occupa solo di simulare $M(\langle M \rangle)$, ed M richiede al più $t_{1.5}(n)$ passi, si avrà che:

$$T(D) = \text{tempo richiesto dalla TM } D = t_{1.5}(n) + \text{overhead di simulazione}$$

Dunque si ha $T(D) \ll t_2(n)$, che implica $L(D) \in \text{DTIME}(t_2(n))$. A questo punto vogliamo mostrare che $L(D) \notin \text{DTIME}(t_1(n))$. Cioè che $\nexists$ TM Q tale. Q decide $L(D)$ in tempo $t_1(n)$. Procediamo per assurdo: supponiamo che $\exists$ TM Q tale. Q decide $L(D)$ in tempo $t_1(n)$. Se questo è vero si ha che sia Q che D decidono $L(D)$. Dunque $\forall x \quad Q(x) = D(x)$.

Diagonalizzazione: Lanciamo $D(\langle Q \rangle)$. Avremo che:

- D simula $Q(\langle Q \rangle)$
- Siccome Q richiede tempo $t_1(n)$ si ha che D riesce ad eseguirlo ed invertire il risultato

Dunque si ha che $Q(x) \neq D(x)$ contraddicendo l'HP iniziale $\Rightarrow \nexists TM Q$ che decide $L(D)$ in tempo $t_1(n) \Rightarrow L(D) \in DTIME(t_2)$ MA $L(D) \notin DTIME(t_1)$.

Enunciare e Dimostrare il Teorema di Gerarchia di Spazio

Teorema: Sia $S_1(n) \geq \log n$ e $S_2(n) = \Omega(S_1(n))$ entrambi spazio costruibili.
Allora $\exists L \in \text{SPACE}(S_2(n))$ ma $L \notin \text{SPACE}(S_1(n))$

Dim. Siano $S_1(n)$ e $S_2(n)$ come definite. Vogliamo mostrare che $\exists L \in \text{SPACE}(S_2(n))$ ma $L \notin \text{SPACE}(S_1(n))$. A tale scopo definiamo una TM D t.c. D richiede spazio $S_2(n)$ ma $L(D)$ non può essere deciso in spazio $S_1(n)$. Nella pratica, la TM D :

- Su input $\langle M \rangle$ con $M \in \text{TM}$
- Sul nastro di lavoro mantiene un contatore per i passi di computazione di M
- Simula $M(\langle M \rangle)$
 - Se $M(\langle M \rangle)$ richiede meno di $2^{O(S_2(n))}$ passi allora D nega il risultato
Cioè $D(\langle M \rangle) = \text{ACC} \Leftrightarrow M(\langle M \rangle) = \text{REJ}$ e viceversa
 - Se $M(\langle M \rangle)$ impiega più di $2^{O(S_2(n))}$ passi, allora $D(\langle M \rangle) = \text{ACC}$.

Costo Spaziale: Poiché $D(\langle M \rangle)$ si occupa solo di simulare $M(\langle M \rangle)$, usando al più $2^{O(S_2(n))}$ passi, si ha che

$S(D) = \text{spazio richiesto dalla simulazione} + \text{overhead di simulazione}$

Dunque si ha $S(D) \ll S_2(n)$, che implica $L(D) \in \text{SPACE}(S_2(n))$. A questo punto vogliamo mostrare che $L(D) \notin \text{SPACE}(S_1(n))$. Cioè che $\nexists \text{TM } Q$ t.c. decide $L(D)$ in spazio $S_1(n)$.

Procediamo per assurdo: supponiamo che $\exists \text{TM } Q$ t.c. Q decide $L(D)$ in spazio $S_1(n)$. Se questo è vero si ha che sia Q che D decidono $L(D)$. Dunque $\forall x \quad Q(x) = D(x)$.

Diagonalizzazione: Lanciamo $D(\langle Q \rangle)$. Avremo che:

- D simula $Q(\langle Q \rangle)$
- Poiché Q richiede spazio $S_1(n)$ si ha che D riesce ad eseguirlo ed invertire il risultato

Dunque si ha che $Q(x) \neq D(x)$ contraddicendo l'HP iniziale $\Rightarrow \nexists TM Q$ tale che DECIDE $L(D)$ in spazio $S_1(n) \Rightarrow L(D) \in SPACE(S_2)$ MA $L(D) \notin SPACE(S_1)$.

NOTA: D conta fino ad un massimo di $2^{O(S_2(n))}$ poiché questo è il numero massimo di iterazioni possibili per M usando $S_2(n)$ spazio. Se M supera questo numero di passi senza terminare significa che è entrata in LOOP.

Utilizzare il Teorema di Gerarchia di Spazio per mostrare che $PSPACE \subsetneq EXPSPACE$

Siccome $PSPACE = \bigcup_{k \in \mathbb{N}} SPACE(n^k)$ e $EXPSPACE = \bigcup_{k \in \mathbb{N}} SPACE(2^{n^k})$, possiamo mostrare che $PSPACE \subsetneq EXPSPACE$ direttamente con lo S.H.T. prendendo:

- $S_1 = n^k$
- $S_2 = 2^{n^k}$

E mostrando che $S_1 \ll S_2$, cioè che $\forall k \lim_{n \rightarrow \infty} \frac{n^k}{2^{n^k}} = 0$

Fornire due definizioni della classe di complessità NP:
tramite Non Determinismo e tramite certificati/verificatori.
Dimostrare che le due definizioni sono equivalenti

① NP è l'insieme dei linguaggi L t.c. L ha un verificatore di tempo polinomiale

Una TM V è un VERIFICATORE per L se:

- V ha input $\langle x, y \rangle$
- $V(\langle x, y \rangle) = \text{Acc} \iff x \in L$

② Sia $\text{NTIME}(f(n))$ è l'insieme di linguaggi L t.c. $\exists \text{NTM } N$ per cui $L(N) = L$
ed N ha tempo $O(f(n))$

Definiamo quindi $\text{NP} = \bigcup_{K \in \mathbb{N}} \text{NTIME}(n^K)$

Dim. $L \in \text{Verifier-NP} \iff L \in \text{NP}$

$\Rightarrow$ Sia L con verificatore V di tempo polinomiale in $|x|$. Dobbiamo definire una NTM N t.c. $L(N) = L$. Come sappiamo, $V(\langle x, y \rangle)$ dove $|y| = O(|x|^K)$. Ovvero, $|y| \leq c \cdot x^K$ con c costante e $K \in \mathbb{N}$. Dunque abbiamo un "bound" sulla lunghezza di y , che ci permette di creare una TM Non-Deterministica che prova tutte le y . Dunque N :

- Indovina non-deterministicamente y (come prima: prova ogni possibile y su un ramo diverso. Poiché sappiamo che ogni y è lunga al più $c \cdot |x|^K = O(|x|^K) = O(n)$ avremo che il tempo di computazione di N è $O(n)$)
- Esegue deterministicamente $V(\langle x, y \rangle)$ in ogni ramo ed accetta sse. $V(\langle x, y \rangle) = \text{Acc}$

Chiaramente $x \in L$ sse. $\exists y$ t.c. $V(\langle x, y \rangle) = \text{Acc}$

⇐ Sia $L \in NP$, ovvero $\exists NTM$ N t.c. $L(N) = L$ e N ha tempo polinomiale. Dobbiamo definire un verificatore V di tempo polinomiale per L .

Idea: Il certificato y sarà l'insieme delle scelte non-deterministiche di N . Questo è rappresentabile con una stringa di lunghezza polinomiale, poiché N ha lunghezza polinomiale.

- Su input $\langle x, y \rangle$ simula deterministicamente $N(x)$ usando le scelte non-deterministiche contenute in y . (cioè y contiene solo un percorso tra tutti quelli non-deterministici. Quando V deve decidere quale scelta non-det fare farà quella che ha fatto y . Cioè sostanzialmente V segue il percorso y .)

Chiaramente $x \in L$ sse. $N(x) = Acc$ (che avviene sse. $\exists y$ t.c. $V(\langle x, y \rangle) = Acc$)
verso di computazione

Fornire due definizioni della classe di complessità NL:
tramite Non Determinismo e tramite certificati/verificatori.
Dimostrare che le due definizioni sono equivalenti

① NL è l'insieme dei linguaggi L t.c. L ha un verificatore log-space

Una TM V è un VERIFICATORE per L se:

- V ha input $\langle x, y \rangle$
- $V(\langle x, y \rangle) = \text{Acc} \iff x \in L$

② Sia $\text{NSPACE}(f(n))$ è l'insieme di linguaggi L t.c. $\exists \text{NTM } N$ per cui $L(N) = L$
ed N ha spazio $O(f(n))$

Definiamo quindi $\text{NL} = \text{NSPACE}(\log n)$

Dim. $L \in \text{Verifier-NL} \iff L \in \text{NL}$

$\Rightarrow$ Sia $L \in \text{Verifier-NL}$, ovvero $\exists$ Verificatore V log-space. Dobbiamo definire una NTM N log-space t.c. $L(N) = L$.

Ci basta considerare N :

- Indovina non-deterministicamente y (1 bit alla volta $\Rightarrow O(1)$)
- Esegue deterministicamente $V(\langle x, y \rangle)$ in ogni ramo ed accetta se. $V(\langle x, y \rangle) = \text{Acc}$
Poiché V è log-space si ha che N richiederà spazio $O(\log n)$ + overhead di simulazione
 $= O(\log n)$

Chiaramente $N(x) = \text{Acc} \iff \exists y$ t.c. $V(\langle x, y \rangle) = \text{Acc}$ e V è log-space.

⇐ Sia $L \in NL$, ovvero $\exists NTM N$ t.c. $L(N) = L$ ed N è log-space. Dobbiamo definire un verificatore V log-space per L .

Idea: Il certificato y sarà l'insieme delle scelte non-deterministiche di N . Tuttavia, siccome $N \in NL$ si ha che richiede spazio $O(\log)$. Dunque il numero massimo di configurazioni per N è $2^{O(\log n)} = O(n) = |y|$.

Dunque y ha lunghezza polinomiale $\Rightarrow$ NON possiamo scriverla sul nastro di lavoro. Dobbiamo utilizzare il nastro di INPUT READ-ONLY. Avremo quindi che V :

- Su input $\langle x, y \rangle$ simula deterministicamente $N(x)$ usando le scelte non-deterministiche contenute in y .
- Ha bisogno solo di memorizzare la conf. corrente. Dunque spazio $O(\log n)$

Chiaramente se $N(x) = \text{Acc}$ allora $\exists y$ t.c. $V(\langle x, y \rangle) = \text{Acc}$.
il ramo seguito da $N(x)$

Definire le classi di complessità P ed NP e la nozione di NP

Completezza

Sia $DTIME(f(n)) = \{L : \exists TM \text{ DETERMINISTICA } M \text{ t.c. } L(M) = L \text{ ed } M \text{ richiede tempo } O(f(n))\}$

$$P = \bigcup_{k \in \mathbb{N}} DTIME(n^k)$$

Sia $NTIME(f(n)) = \{L : \exists NTM M \text{ t.c. } L(M) = L \text{ ed } M \text{ richiede tempo } O(f(n))\}$

$$NP = \bigcup_{k \in \mathbb{N}} NTIME(n^k)$$

Alternativa:

$$\begin{aligned} NP &= \{L : \exists V \in TM \text{ t.c. } V \text{ è verificatore per } L \text{ e richiede tempo polinomiale}\} = \\ &= \{L : \exists V \in TM \text{ t.c. } x \in L \iff \exists y : V(\langle x, y \rangle) = Acc\} \end{aligned}$$

Inoltre avremo che L è NP-Completo se $L \in NP$ e $\forall A \in NP \quad A \leq_m^P L$ (cioè L è NP Hard)

Sia L un Linguaggio PSPACE Completo.
Mostrare che se $L \in NP$ allora $NP = PSPACE$

Per definizione, se L è PSPACE-Completo si ha che $L \in PSPACE$ e $\forall A \in PSPACE \ A \leq_m^P L$.
Ma, per uno dei teoremi sulle mapping reduction, si ha che:

- Se $A \leq_m^P L$ ed $L \in NP$ allora $A \in NP$

Dunque, siccome $L \in NP$ per ipotesi:

$$L \text{ PSPACE-Completo} \Rightarrow \forall A \in PSPACE \ A \leq_m^P L \Rightarrow A \in NP \Rightarrow PSPACE \subseteq NP$$

Inoltre, poiché sappiamo che il tempo limita lo spazio, si ha anche $NP \subseteq PSPACE$.
Possiamo quindi concludere che $NP = PSPACE$.

Definire la classe di complessità coNP ed il problema UNSAT. Dimostrare che UNSAT è coNP Completo

$$\text{def } \text{coNP} = \{L : \overline{L} \in \text{NP}\}$$

$$\text{def } \text{UNSAT} = \{\langle \phi \rangle : \langle \phi \rangle \notin \text{SAT}\} = \overline{\text{SAT}}$$

Dim. Mostrare che UNSAT è coNP-Completo significa mostrare che:

$$\textcircled{1} \text{ UNSAT} \in \text{coNP}$$

$$\textcircled{2} \forall L \in \text{coNP} \quad L \leq_m^P \text{UNSAT}$$

$$\textcircled{1} \text{ UNSAT} \in \text{coNP} \iff \overline{\text{UNSAT}} \in \text{NP}$$

Ma $\overline{\text{UNSAT}} = \text{SAT}$, che sappiamo essere NP-Completo. Dunque si ha $\text{SAT} \in \text{NP}$, che di conseguenza implica $\text{UNSAT} \in \text{coNP}$.

$$\textcircled{2} \text{ Poiché sappiamo che SAT è NP-Completo si ha che } \text{SAT} \in \text{NP} \text{ e } \forall L \in \text{NP} \quad L \leq_m^P \text{SAT}.$$

Inoltre, poiché le mapping reduction sono chiuse per il complemento, possiamo negare da entrambi i lati, ed otterremo che:

$$\forall L \in \text{NP} \quad \overline{L} \leq_m^P \overline{\text{SAT}}$$

Ma $\overline{\text{SAT}} = \text{UNSAT}$ e $\forall L \in \text{NP} \quad \overline{L} \in \text{coNP}$. Otteniamo quindi:

$$\forall L \in \text{coNP} \quad L \leq_m^P \text{UNSAT}$$

Dimostrare che se $P = NP$ allora $EXP = NEXP$

Procediamo per doppia inclusione:

$EXP \subseteq NEXP$ Questo lato è banale per definizione. Se $L \in EXP$ significa che $\exists M \in TM$ deterministica t.c. $L(M) = L$ ed M richiede tempo $O(2^{n^k})$. Ma poiché ogni TM deterministica è anche Non-Deterministica (con un solo ramo di computazione) avremo anche che $L \in NEXP$.

$NEXP \subseteq EXP$ Procediamo facendo Padding.

Sia $L \in NEXP$, allora si ha che $\exists M \in TM$ t.c. $L(M) = L$ ed M richiede tempo $O(2^{n^k})$. Costruiamo ora $L' = \{w : w = x\#^k \text{ con } x \in L \text{ e } k = 2^{|x|^c}\}$ ed una TM M' t.c.

- Su input $w = x\#^k$ con $k = 2^{|x|^c}$
 - se w NON è in questa forma allora $M'(w) = REJ$ subito
- Simula $M(x)$:
 - se $M(x) = ACC$ allora $M'(w) = ACC$
 - se $M(x) = REJ$ allora $M'(w) = REJ$

$$|w| = |x| + |\#^{2^{|x|^c}}| = |x| + 2^{|x|^c} = O(2^{|x|^c})$$

Sia $N =$ la grandezza dell'input di $M' \downarrow |w| = O(2^{|x|^c})$. Avremo che $Tempo(M') = O(N)$ è POLINOMIALE. Dunque $L(M') = L' \in NP$.

Ma per ipotesi sappiamo che $P = NP$. Dunque $L' \in P$ ed $\exists M'' \in TM$ Deterministica t.c. $L(M'') = L'$. Definiamo ora M''' :

- Su input x
- Aggiunge padding $\#^k$ con $k = 2^{|x|^c}$ ad x . Otteniamo quindi $w = x\#^k$

- Simula $M''(w)$:
 - se $M''(w) = \text{ACC}$ allora $M'''(x) = \text{ACC}$
 - se $M''(w) = \text{REJ}$ allora $M'''(x) = \text{REJ}$

Sia $n = |x|$ la grandezza dell'input $M''' = |x|$. Si avrà che il costo temporale di M''' è dato da:

- Tempo per aggiungere padding = $O(2^{|x|}) = O(2^n)$
 +
 (poiché aggiungiamo $\#^k$ con $k = 2^{|x|}$)
- Tempo per simulare $M''(w) = O(N) = O(2^{|x|}) = O(2^n)$

Dunque $\text{Tempo}(M''') = O(2^n)$. Inoltre, poiché M''' si occupa solo di simulare M'' , ed M'' è Deterministico, si ha che anche M''' è Deterministico. Dunque $L(M''') \in \text{EXP}$

Dimostrare che se f, g e t sono funzioni tempo costruttibili e $\text{DTIME}(f(n)) = \text{DTIME}(g(n))$ allora $\text{DTIME}(f(t(n))) = \text{DTIME}(g(t(n)))$

① Procediamo con doppia inclusione

$\subseteq$ Prendiamo $L \in \text{DTIME}(f(t(n)))$, questo significa che $\exists M \in \text{TM}$ tale che $L(M) = L$ e M richiede tempo $f(t(n))$

Definiamo ora $L' = \{ x \#^k : x \in L \text{ e } k = t(|x|) - |x| \}$ ed M' tale.

- Su input $w = x \#^k$ con $k = t(|x|) - |x|$. Se w non è in questa forma allora $M'(w) = \text{REJ}$ subito
- Simula $M(x)$. Cioè:
 - $M'(w) = \text{ACC}$ sse. $M(x) = \text{ACC}$
 - $M'(w) = \text{REJ}$ sse. $M(x) = \text{REJ}$

È evidente che $L(M') = L'$. Sia ora $N = |w| = t(|x|)$. Avremo che:

$$\text{Tempo}(M') = \text{simulazione di } M(x)$$

Ma siccome per ipotesi si ha che M richiede tempo $f(t(n))$ possiamo dire che

$$\text{Tempo}(M') = O(f(t(n))) = O(f(N))$$

Ma questo comporta $L(M') = L \in \text{DTIME}(f(n))$ che, per ipotesi, implica $M' \in \text{DTIME}(g(n))$.

Costruiamo ora una TM M'' tale.

- Su input x , aggiunge padding $\#$ per ottenere $w = x \#^k$, con $k = t(|x|) - |x|$
- Simula $M'(w)$. Cioè:

- $M''(x) = \text{ACC}$ sse. $M'(w) = \text{ACC}$
- $M''(x) = \text{REJ}$ sse. $M'(w) = \text{REJ}$

Anche qui, è evidente che $L(M'') = L$. Inoltre:

$$\text{Tempo}(M'') = \text{tempo per fare padding} + \text{simulazione di } M'(w)$$

Ma fare padding ci costa $O(t(n))$, mentre la simulazione ci costa tanto quanto M' . E siccome $M' \in \text{DTIME}(g(n))$, con $n = |w| = t(|x|)$, si ottiene che:

$$\text{Tempo } M'' = O(t(n)) + O(g(t(n))) = O(g(t(n)))$$

concludiamo quindi che $L(M'') = L \in \text{DTIME}(g(t(n)))$

$\square$ Per simmetria lo stesso ragionamento vale scambiando f e g .